

Library Management System

Complete Project Guide — Java + MySQL + JDBC + IntelliJ IDEA

This guide explains the complete beginner-friendly Library Management System, including project structure, database setup, Java classes, operations, testing, demonstration flow, architecture, and viva questions.

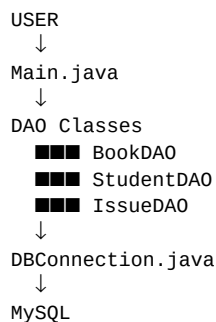
1. Project Overview

Project Name: Library Management System

Technologies: Java, MySQL, JDBC, IntelliJ IDEA, Maven

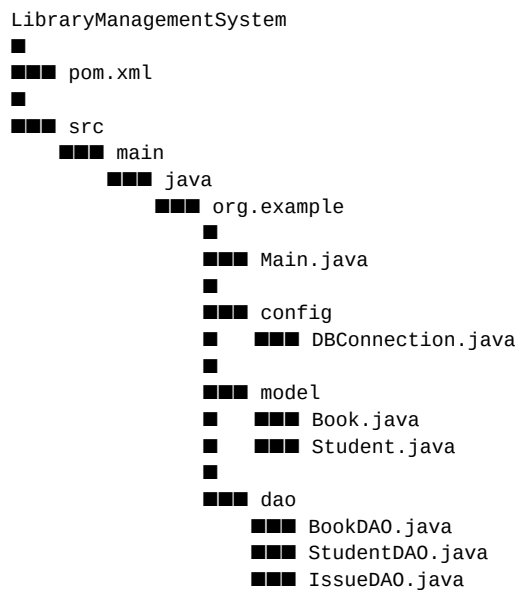
Main Purpose: Manage books, students, book issuing, book returning, searching books, and database records.

2. Overall Architecture



Main.java provides the menu and accepts user input. Model classes represent data. DAO classes perform database operations. DBConnection.java connects Java to MySQL.

3. IntelliJ Project Structure



4. Maven Dependency

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
```

```
<version>9.4.0</version>
</dependency>
```

Maven downloads the MySQL JDBC driver required for Java to communicate with MySQL.

5. Create MySQL Database

```
CREATE DATABASE librarydb;

USE librarydb;
```

6. Create Books Table

```
CREATE TABLE books (
    book_id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(100) NOT NULL,
    author VARCHAR(100) NOT NULL,
    quantity INT NOT NULL
);
```

7. Create Students Table

```
CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100)
);
```

8. Create Issued Books Table

```
CREATE TABLE issued_books (
    issue_id INT PRIMARY KEY AUTO_INCREMENT,
    book_id INT,
    student_id INT,
    issue_date DATE,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES books(book_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id)
);
```

9. DBConnection.java

```
package org.example.config;

import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnection {

    private static final String URL =
        "jdbc:mysql://localhost:3306/librarydb";

    private static final String USER = "root";

    private static final String PASSWORD =
        "YOUR_PASSWORD";

    public static Connection getConnection() {
        try {
            Connection connection =
                DriverManager.getConnection(URL, USER, PASSWORD);

            System.out.println("Database Connected Successfully!");
            return connection;

        } catch (Exception e) {
            System.out.println("Database Connection Failed!");
            e.printStackTrace();
            return null;
        }
    }
}
```

```
    }  
  }  
}
```

Replace YOUR_PASSWORD with your MySQL password. Do not share your real database password publicly.

10. Book.java

```
package org.example.model;  
  
public class Book {  
  
    private int bookId;  
    private String title;  
    private String author;  
    private int quantity;  
  
    public Book() {  
    }  
  
    public Book(String title, String author, int quantity) {  
        this.title = title;  
        this.author = author;  
        this.quantity = quantity;  
    }  
  
    public int getBookId() {  
        return bookId;  
    }  
  
    public void setBookId(int bookId) {  
        this.bookId = bookId;  
    }  
  
    public String getTitle() {  
        return title;  
    }  
  
    public void setTitle(String title) {  
        this.title = title;  
    }  
  
    public String getAuthor() {  
        return author;  
    }  
  
    public void setAuthor(String author) {  
        this.author = author;  
    }  
  
    public int getQuantity() {  
        return quantity;  
    }  
  
    public void setQuantity(int quantity) {  
        this.quantity = quantity;  
    }  
}
```

11. Student.java

```
package org.example.model;  
  
public class Student {  
  
    private int studentId;  
    private String name;  
    private String email;  
  
    public Student() {  
    }  
  
    public Student(String name, String email) {
```

```

        this.name = name;
        this.email = email;
    }

    public int getStudentId() {
        return studentId;
    }

    public void setStudentId(int studentId) {
        this.studentId = studentId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}

```

12. BookDAO.java

```

package org.example.dao;

import org.example.config.DBConnection;
import org.example.model.Book;

import java.sql.Connection;
import java.sql.PreparedStatement;

public class BookDAO {

    public void addBook(Book book) {
        String sql =
            "INSERT INTO books(title, author, quantity) VALUES (?, ?, ?)";

        try {
            Connection con = DBConnection.getConnection();
            PreparedStatement ps = con.prepareStatement(sql);

            ps.setString(1, book.getTitle());
            ps.setString(2, book.getAuthor());
            ps.setInt(3, book.getQuantity());

            ps.executeUpdate();

            System.out.println("Book added successfully!");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public void viewBooks() {
        String sql = "SELECT * FROM books";

        try {
            Connection con = DBConnection.getConnection();
            PreparedStatement ps = con.prepareStatement(sql);
            var rs = ps.executeQuery();

            while (rs.next()) {
                System.out.println("Book ID: " + rs.getInt("book_id"));
                System.out.println("Title: " + rs.getString("title"));
            }
        }
    }
}

```

```

        System.out.println("Author: " + rs.getString("author"));
        System.out.println("Quantity: " + rs.getInt("quantity"));
        System.out.println("-----");
    }

    } catch (Exception e) {
        e.printStackTrace();
    }
}

public void searchBook(String title) {
    String sql =
        "SELECT * FROM books WHERE title LIKE ?";

    try {
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql);

        ps.setString(1, "%" + title + "%");

        var rs = ps.executeQuery();

        while (rs.next()) {
            System.out.println("Book ID: " + rs.getInt("book_id"));
            System.out.println("Title: " + rs.getString("title"));
            System.out.println("Author: " + rs.getString("author"));
            System.out.println("Quantity: " + rs.getInt("quantity"));
            System.out.println("-----");
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

13. StudentDAO.java

```

package org.example.dao;

import org.example.config.DBConnection;
import org.example.model.Student;

import java.sql.Connection;
import java.sql.PreparedStatement;

public class StudentDAO {

    public void addStudent(Student student) {
        String sql =
            "INSERT INTO students(name, email) VALUES (?, ?)";

        try {
            Connection con = DBConnection.getConnection();
            PreparedStatement ps = con.prepareStatement(sql);

            ps.setString(1, student.getName());
            ps.setString(2, student.getEmail());

            ps.executeUpdate();

            System.out.println("Student added successfully!");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public void viewStudents() {
        String sql = "SELECT * FROM students";

        try {
            Connection con = DBConnection.getConnection();

```

```

        PreparedStatement ps = con.prepareStatement(sql);
        var rs = ps.executeQuery();

        while (rs.next()) {
            System.out.println(
                "Student ID: " + rs.getInt("student_id"));
            System.out.println(
                "Name: " + rs.getString("name"));
            System.out.println(
                "Email: " + rs.getString("email"));
            System.out.println("-----");
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

14. IssueDAO.java

```

package org.example.dao;

import org.example.config.DBConnection;

import java.sql.Connection;
import java.sql.PreparedStatement;

public class IssueDAO {

    public void issueBook(int bookId, int studentId) {
        try {
            Connection con = DBConnection.getConnection();

            String issueSql =
                "INSERT INTO issued_books(book_id, student_id, issue_date) " +
                "VALUES (?, ?, CURDATE())";

            PreparedStatement issuePs =
                con.prepareStatement(issueSql);

            issuePs.setInt(1, bookId);
            issuePs.setInt(2, studentId);
            issuePs.executeUpdate();

            String updateSql =
                "UPDATE books SET quantity = quantity - 1 " +
                "WHERE book_id = ? AND quantity > 0";

            PreparedStatement updatePs =
                con.prepareStatement(updateSql);

            updatePs.setInt(1, bookId);
            updatePs.executeUpdate();

            System.out.println("Book issued successfully!");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public void returnBook(int issueId) {
        try {
            Connection con = DBConnection.getConnection();

            String findSql =
                "SELECT book_id FROM issued_books " +
                "WHERE issue_id = ? AND return_date IS NULL";

            PreparedStatement findPs =
                con.prepareStatement(findSql);

            findPs.setInt(1, issueId);

```

```

        var rs = findPs.executeQuery();

        if (rs.next()) {
            int bookId = rs.getInt("book_id");

            String returnSql =
                "UPDATE issued_books SET return_date = CURDATE() " +
                "WHERE issue_id = ?";

            PreparedStatement returnPs =
                con.prepareStatement(returnSql);

            returnPs.setInt(1, issueId);
            returnPs.executeUpdate();

            String updateSql =
                "UPDATE books SET quantity = quantity + 1 " +
                "WHERE book_id = ?";

            PreparedStatement updatePs =
                con.prepareStatement(updateSql);

            updatePs.setInt(1, bookId);
            updatePs.executeUpdate();

            System.out.println("Book returned successfully!");
        } else {
            System.out.println(
                "Invalid Issue ID or book already returned!");
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

15. Final Main.java

```

package org.example;

import org.example.dao.BookDAO;
import org.example.dao.StudentDAO;
import org.example.dao.IssueDAO;
import org.example.model.Book;
import org.example.model.Student;

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        BookDAO bookDAO = new BookDAO();
        StudentDAO studentDAO = new StudentDAO();
        IssueDAO issueDAO = new IssueDAO();

        while (true) {

            System.out.println("\n=====");
            System.out.println("        LIBRARY MANAGEMENT SYSTEM");
            System.out.println("=====");

            System.out.println("1. Add Book");
            System.out.println("2. View Books");
            System.out.println("3. Search Book");
            System.out.println("4. Add Student");
            System.out.println("5. View Students");
            System.out.println("6. Issue Book");
            System.out.println("7. Return Book");

```

```

System.out.println("8. Exit");

System.out.print("Enter your choice: ");
int choice = sc.nextInt();
sc.nextLine();

switch (choice) {

    case 1:
        System.out.print("Enter book title: ");
        String title = sc.nextLine();

        System.out.print("Enter author: ");
        String author = sc.nextLine();

        System.out.print("Enter quantity: ");
        int quantity = sc.nextInt();

        Book book =
            new Book(title, author, quantity);

        bookDAO.addBook(book);
        break;

    case 2:
        bookDAO.viewBooks();
        break;

    case 3:
        System.out.print(
            "Enter book title to search: ");
        String searchTitle = sc.nextLine();

        bookDAO.searchBook(searchTitle);
        break;

    case 4:
        System.out.print(
            "Enter student name: ");
        String name = sc.nextLine();

        System.out.print(
            "Enter student email: ");
        String email = sc.nextLine();

        Student student =
            new Student(name, email);

        studentDAO.addStudent(student);
        break;

    case 5:
        studentDAO.viewStudents();
        break;

    case 6:
        System.out.print("Enter Book ID: ");
        int bookId = sc.nextInt();

        System.out.print("Enter Student ID: ");
        int studentId = sc.nextInt();

        issueDAO.issueBook(bookId, studentId);
        break;

    case 7:
        System.out.print("Enter Issue ID: ");
        int issueId = sc.nextInt();

        issueDAO.returnBook(issueId);
        break;

    case 8:
        System.out.println(

```



```

        "Thank you for using Library Management System!");

        sc.close();
        return;

    default:
        System.out.println("Invalid choice!");
    }
}
}
}

```

16. Final Menu

```

=====
      LIBRARY MANAGEMENT SYSTEM
=====

1. Add Book
2. View Books
3. Search Book
4. Add Student
5. View Students
6. Issue Book
7. Return Book
8. Exit

Enter your choice:

```

17. Complete Operation Flow

```

START
↓
Run Main.java
↓
Display Menu
↓
User selects operation
↓
Book / Student / Issue DAO
↓
DBConnection
↓
MySQL
↓
Result displayed
↓
Return to Menu
↓
Exit

```

18. Database Relationship

```

BOOKS
|
| book_id
↓
ISSUED_BOOKS
↑
| student_id
|
STUDENTS

```

The issued_books table connects a particular book with a particular student.

19. What Each File Does

File	Purpose
------	---------

Main.java	Main menu and user input
DBConnection.java	Connects Java to MySQL
Book.java	Stores book information
Student.java	Stores student information
BookDAO.java	Add, view and search books
StudentDAO.java	Add and view students
IssueDAO.java	Issue and return books
pom.xml	Maven dependency management

20. How to Demonstrate the Project

- Start MySQL Server.
- Open the project in IntelliJ IDEA.
- Run Main.java.
- Show the Library Management System menu.
- Demonstrate Add Book.
- Demonstrate View Books.
- Demonstrate Search Book.
- Demonstrate Add Student.
- Demonstrate View Students.
- Demonstrate Issue Book and show quantity decreasing.
- Open MySQL Workbench and show books, students and issued_books tables.
- Demonstrate Return Book and show return_date and quantity update.
- Choose Exit.

21. Example Demonstration

```

1. Add Book
Enter book title: Java Programming
Enter author: James Gosling
Enter quantity: 5

2. View Books
Book ID: 1
Title: Java Programming
Author: James Gosling
Quantity: 5

4. Add Student
Enter student name: Rahul
Enter student email: rahul@gmail.com

6. Issue Book
Enter Book ID: 1

```

Enter Student ID: 1

7. Return Book
Enter Issue ID: 1

22. Useful MySQL Commands

```
USE librarydb;  
  
SELECT * FROM books;  
  
SELECT * FROM students;  
  
SELECT * FROM issued_books;  
  
SHOW TABLES;
```

23. Viva Questions and Answers

Q: What is the purpose of this project?

A: It manages books, students, issuing and returning of books using Java and MySQL.

Q: Why MySQL?

A: MySQL stores the project data permanently in a relational database.

Q: Why Java?

A: Java provides object-oriented programming and is suitable for database applications.

Q: What is JDBC?

A: JDBC stands for Java Database Connectivity and allows Java to communicate with a database.

Q: What is DAO?

A: DAO stands for Data Access Object. DAO classes contain database operations.

Q: What happens when a book is issued?

A: An issue record is created and the available quantity is decreased.

Q: What happens when a book is returned?

A: The return date is stored and the available quantity is increased.

Q: Why primary keys?

A: They uniquely identify records in a table.

Q: Why foreign keys?

A: They connect records between related tables.

24. Recommended Improvements Before Submission

- Prevent issuing a book when quantity is zero.
- Check whether the book exists before issuing.
- Check whether the student exists before issuing.
- Prevent returning an already returned book.
- Show a clear message when a search finds no book.
- Validate empty names, titles and invalid quantities.
- Optionally add an admin login.

- Optionally add a fine calculation for late returns.
- Use database transactions for issue/return operations in a more advanced version.

25. Short Presentation Script

My project is a Library Management System developed using Java and MySQL. Java is used for the application logic and MySQL is used for data storage. JDBC is used to connect Java with MySQL. The system allows the librarian to add, view and search books, add and view students, issue books and return books. DAO classes are used to separate database operations from the main program. The project is a console-based application and all important records are stored in the MySQL database.